UNIVERSITÉ DU LITTORAL CÔTE D'OPALE

Licence 3 Informatique
Informatique Théorique 3
2024-2025

TP2 — Grammaires

Ce TP porte sur les expressions régulières et les grammaires pour l'analyse syntaxique ascendante. Dans ce cadre, vous utiliserez FLEX et YACC en python afin d'exprimer les mots et phrases acceptés.
L'objectif de ce TP est de reconnaître des instructions permettant de faire des calculs. Plus de détails sont donnés dans la partie 1.
Les fichiers pour FLEX et YACC en python sont fournis dans le matériel supplémentaire du TP, en plus du fichier *tp2.py* dans lequel vous produirez votre code. Le fichier *test.txt* pourra être modifié pour tester différentes instructions.

A Instructions à reconnaître

Cette partie indique juste les instructions qui devraient être reconnues à la fin par votre programme mais ne contient pas de questions. Elle contient indique cependant par quoi commencer.
L'objectif est de reconnaître des expressions arithmétiques valides et de produire un arbre syntaxique associé aux expressions reconnues. Pour simplifier, seule la reconnaissance d'une instruction à la fois sera possible (une ligne). Cette expression peut cependant être très longue, du moment qu'elle est valide.
Il faudra reconnaître les opérations suivantes :

- l'addition
- la soustraction
- la multiplication
- la division

Les opérations pouront être utilisées au sein d'expressions contenant :

- des parenthèses,
- des entiers,
- des floats,
- des variables.

On pourra aussi reconnaître l'assignation d'une valeur à une variable. Cette assignation se fera simplement avec l'opérateur "=" mais pourra être suivi d'une expression.
Voici quelques exemples d'expressions pouvant être reconnues (une à la fois) :

```
1+3
5-4*12
tmp+4
-2
Variable = ((-3+5.4)/8)*7
```

Pour l'expression $5+3+8$, l'exécution de votre code devrait afficher l'arbre syntaxique suivant ('binop', '+', ('binop', '+', ('number', 5), ('number', 3)), ('number', 8)).

Dans un premier temps, vous travaillerez uniquement avec des entiers. Il est conseillé d'ajouter les différentes opérations progressivement avant d'intégrer les floats et les variables.

Vous commencerez par :

— la reconnaissance d'un nombre entier positif,
— puis la reconnaissance des additions et soustractions,
— puis la reconnaissance de la multiplication et la division,
— puis la reconnaissance des entiers négatifs,
— puis la reconnaissance des parenthèses,
— puis la gestion des floats,
— et enfin la gestion des variables, incluant leur assignation.

B FLEX et expressions régulières

FLEX sera utilisé pour définir les tokens de la grammaire. Parmi les tokens, on retrouve les symboles terminaux ainsi que les expressions régulières qui permettront la reconnaissance de nombres et de chaînes de caractères.

Tous les tokens doivent être listés dans la variable *tokens*. Par exemple, pour 2 tokens nommés *NOMTOKEN1* et *NOMTOKEN2*, il faudra les lister de la manière suivante :

```
tokens = ("NOMTOKEN1", "NOMTOKEN2")
```

Une fois les tokens listés, il faut les définir. Dans la **majorité des cas**, cela se fait au moyen d'une variable à laquelle on assigne une expression régulière. Par exemple, pour un token nommé *AMINUSCULE*, qui reconnaît la lettre "a" en minuscule, cela se fait de la manière suivante :

```
t_AMINUSCULE = r'a'
```

Attention, il est impératif que :

— la variable soit nommée avec un "t" suivi d'un _ et du nom du token,
— l'expression régulière soit précisée après la lettre r,
— l'expression régulière soit précisée entre simple guillemets.

Note : dans certains cas, il sera nécessaire d'échapper certains caractères des expressions régulières.

Dans certains cas, par exemple celui des nombres, la reconnaissance de l'expression régulière est gérée dans une fonction. Dans le cas des nombres, l'objectif est de spécifier en python le type de l'expression régulière reconnue. Cela se fait de la manière suivante :

```
def t_NOMTOKEN(t):
  r""" expression reguliere """"
  #instructions pour traitement
  return t
```

où "expression reguliere" est remplacé par l'expression régulière permettant de reconnaître le token et où des instructions de traitement sont écrites à la place du commentaire.

C YACC et grammaires

La définition des règles de la grammaire se fait après la gestion du lexer.

Attention, contrairement aux grammaires vues en cours, il sera nécessaire de décomposer certaines règles de la grammaire en faisant intervenir plus de non-terminaux. Cela permet de gérer

les règles de priorité : une règle écrite en dernier est traitée prioritairement par rapport aux autres.
Une ou plusieurs règles sont reconnues et gérées au sein d'une même fonction (séquence). Dans sa forme la plus simple, une séquence est définie de la manière suivante :

```
1 def p_regle1(p):
2   """
3   regle1 : regle2
4   """
5   p[0] = p[1]
```

où *regle1* correspond au nom de la règle traitée, la règle est spécifiée ligne 3, *p[0]* correspond à la partie gauche de la règle (avant la flèche) et la partie après p[0] correspond à la valeur ou règle en laquelle p[0] s'est dérivé. Dans l'exemple, p[0] prend la valeur de p[1], qui correspond à la valeur de regle2.

Il faudra parfois plusieurs séquences (plusieurs fonctions) pour traiter les différentes règles associées à un même non-terminal. Cela peut être le cas quand une des règles à droite contient plus d'élements qu'une autre, ou des éléments de types différents. Par exemple, ci-dessous, pour $regle1 \rightarrow regle2|number$, avec *regle2* un non-terminal et *NB* un token :

```
1 def p_regle1(p):
2   """
3   regle1 : regle2
4   """
5   p[0] = p[1]
6
7 def p_regle1_nb(p):
8   """
9   regle1 : NB
10  """
11  p[0] = ("number", p[1])
```

où pour p[0] on spécifie que la valeur de p[1] correspond à un nombre.
Plusieurs possibilités existent pour le traitement de p[0]. Par exemple, dans le cas d'une séquence contenant 3 éléments dont un opérateur binaire, on peut avoir p[0] = ("binop", p[2], p[1], p[3]), où p[2] contient l'opérateur binaire et nécessite trois éléments. Parmi les autres possibilités utiles dans ce TP on trouve :

- "unary" pour l'opérateur unaire, nécessite trois éléments,
- "binary" pour l'égalité, nécessite trois éléments,
- "grouped" pour les parenthèses, nécessite un élément (mais trois dans la règle)
- "name" pour les identifiants, nécessite un élément

D Bonus

Si toutes les étapes précédentes sont gérées, vous pouvez travailler sur la traduction des instructions ne contenant pas de variables en code python. Pour la traduction, l'arbre d'analyse doit être parcouru en profondeur.